MODULE 23 · DOMAIN 3 - DEPLOYMENT, PROVISIONING, AND AUTOMATION

Event-Driven Operations

Lambda concurrency and retries, S3 notifications, Step Functions, service quotas and tagging.

DOMAIN 3 of 5 EXAM WEIGHT 22% MODULE 23 of 35

admin@sthithapragna.com

Serverless removes the servers and keeps the limits. Concurrency, retries and quotas are where event-driven systems actually fail.

Three invocation models, three failure behaviours

MODEL	WHAT HAPPENS ON ERROR
Synchronous (API Gateway, ALB)	No retry. The caller sees the error and decides
Asynchronous (S3, SNS, EventBridge)	Two retries, then a destination or DLQ, or dropped
Event source mapping (SQS)	Visibility timeout and maxReceiveCount, then the queue's DLQ
Event source mapping (Kinesis, DDB Streams)	Retries in order, and can block the shard

ON THE EXAM

The stream row is the dangerous one. One poison-pill record can stall a shard for the whole retention period.

Asynchronous failure handling

- Two retries, with delays. **Maximum event age** defaults to **6 hours**; after that the event is discarded regardless.
- With **no destination and no DLQ, a failed event is simply lost**. There is no error anywhere except the metric.
- **On-failure destinations** are richer than a DLQ: they include the **request and response context**, not just the payload.
- Destinations also support **on-success**, and can target SNS, SQS, Lambda or EventBridge. A DLQ is failure-only, and SNS or SQS only.

> ON THE EXAM
>
> Destinations superseded the DLQ for asynchronous invocation. If a question offers both, destinations give you more and cost no more.

The concurrency model

- One **account-wide concurrency pool per Region**, defaulting to **1,000**. Every function shares it.
- One busy function can consume most of the pool, and every other function in the account fights over what is left.
- **Reserved concurrency** carves out a guaranteed slice for one function. It is both a **floor and a ceiling**.
- Reserving capacity **removes it from the shared pool**, so an over-reserved function starves everything else.

> ON THE EXAM
>
> Reserved concurrency set to zero is the emergency stop. It guarantees the function cannot run at all, which is occasionally exactly what you need.

Reserved or provisioned concurrency?

RESERVED CONCURRENCY

How many may run

- Guarantees a slice of the account pool
- Also caps the function at that number
- Protects other functions from this one
- No additional charge

PROVISIONED CONCURRENCY

How many are already warm

- Pre-initialised environments, no cold start
- Charged for the environments, whether used or not
- Auto-scalable with Application Auto Scaling on the alias
- Set within the reserved limit if both are used

THE TIEBREAKER

Reserved solves throttling and noisy neighbours. Provisioned solves cold-start latency. They answer completely different complaints.

Throttled with capacity to spare

- There is a second limit: the **scaling rate**. Lambda creates **1,000 new execution environments every 10 seconds**, per function, per Region.
- A sharp spike can be **throttled while the account pool is nowhere near full**. The dashboard shows plenty of capacity.
- The signature is a **stair-step ConcurrentExecutions graph** with a matching spike of Throttles.
- **ClaimedAccountConcurrency** shows how full the pool is. The scaling rate governs how fast you may fill it.

> ON THE EXAM
>
> "We were throttled but only using 30 percent of our concurrency" is the scaling rate, not the pool. Two limits, one metric people watch.

Protecting what is downstream

- Lambda scales far faster than most databases. A thousand concurrent functions means a thousand connection attempts.
- **RDS Proxy** pools those connections, as module ten covered. That is the relational answer.
- **Reserved concurrency** is the blunt answer: cap the function so the database is never asked for more than it can take.
- For SQS sources, **MaximumConcurrency on the event source mapping** caps concurrency without reserving account capacity.

> ON THE EXAM
>
> The event source mapping's MaximumConcurrency is the precise tool. Reserved concurrency also removes capacity from the shared pool; this does not.

Cold starts

- **SnapStart** takes a snapshot of the initialised environment and restores from it. Java, Python and .NET.
- **SnapStart and provisioned concurrency cannot be combined.** You choose one.
- SnapStart costs less and gives sub-second starts. Provisioned concurrency costs more and works for every runtime.
- **Attaching a function to a VPC** historically added start-up time. It also means the function needs NAT or endpoints to reach AWS APIs.

> ON THE EXAM
>
> Cold starts matter most on synchronous, user-facing paths. An asynchronous event handler rarely needs either mechanism.

S3 Event Notifications

- Destinations: **SNS, SQS, Lambda**, or **EventBridge**. Filter by object key **prefix and suffix**.
- **Overlapping prefix or suffix configurations for the same event type are not allowed.** This limitation forces awkward designs.
- Sending to **EventBridge** instead removes that limitation and gives far richer filtering, plus every EventBridge target.
- EventBridge also carries **more S3 event types** and integrates with archive, replay and cross-account buses.

> ON THE EXAM
>
> "Several teams need to react to the same uploads with different filters" is the overlap limitation. Route through EventBridge.

Step Functions: Standard or Express?

STANDARD

Long, durable, auditable

- Up to one year of execution
- Exactly-once execution semantics
- Full execution history in the console
- Priced per state transition

EXPRESS

Short, fast, high volume

- Up to five minutes
- At-least-once for async; at-most-once for sync
- History goes to CloudWatch Logs, not the console
- Priced per invocation and duration

THE TIEBREAKER

Volume decides it. Thousands of short executions per second is Express; an orchestration that waits hours for approval is Standard.

Step Functions for operations

- **Retry** and **Catch** are declarative: backoff, jitter, maximum attempts and error-specific handling, with no code.
- **Request-response** calls and returns. **.sync** starts a job and waits for it to finish. **.waitForTaskToken** pauses until something calls back.
- The callback pattern is how a workflow waits for **a human approval** or an external system, for up to a year.
- **Distributed Map** reads a large dataset from S3 and runs up to **10,000 parallel child executions**.

> ON THE EXAM
>
> Retry and Catch in the state machine replace error-handling code in every Lambda function. That is the operational argument for Step Functions.

At-least-once means design for duplicates

- **SQS standard queues, SNS, EventBridge and Lambda retries can all deliver the same event more than once.** That is normal, not a fault.
- **Idempotency** means processing the same event twice has the same effect as processing it once.
- The usual mechanism: an **idempotency key** from the event, recorded in DynamoDB with a TTL, checked before doing the work.
- **FIFO queues and FIFO topics** give exactly-once processing and ordering, at much lower throughput.

> ON THE EXAM
>
> Module eight said an automated fix must be safe to run twice. This is the same requirement, and here it is unavoidable rather than advisable.

Queues and topics, operationally

- **Visibility timeout** must exceed your processing time, or the same message is delivered again while you are still working on it.
- **maxReceiveCount** plus a dead-letter queue is how a poison message stops blocking the queue.
- **DLQ redrive** moves messages back to the source queue after you fix the cause. No custom script needed.
- **Alarm on ApproximateAgeOfOldestMessage**, not just queue depth. A shallow queue whose oldest message is hours old is stuck.

> ON THE EXAM
>
> Queue depth alone is a poor signal. A queue can be nearly empty and completely stalled on one message nobody can process.

SNS operationally

- **Fan-out**: one message to many subscribers. The classic pattern is SNS to several SQS queues, one per consuming team.
- **Subscription filter policies** let each subscriber receive only the messages it cares about, evaluated by SNS rather than the consumer.
- Each subscription can have its own **delivery retry policy** and its own **dead-letter queue**.
- A failed HTTP subscriber does not block the others. Delivery is per subscription, and so is failure.

> ON THE EXAM
>
> SNS to SQS is the durability pattern. The topic fans out; the queues absorb bursts and retain messages if a consumer is down.

SQS, SNS or EventBridge?

SERVICE	REACH FOR IT WHEN
Amazon SQS	One consumer, buffered. Absorb bursts, retain work, control the rate
Amazon SNS	Many subscribers, one message, very high throughput and low latency
Amazon EventBridge	Routing on event content, AWS service events, archive and replay
SNS to SQS	Fan-out plus durability. Each consumer gets its own buffered copy

ON THE EXAM

A queue decouples in time; a topic decouples in number; a bus decouples in content. Most real designs use more than one.

Service Quotas

- Quotas are **per account and per Region**. Some are adjustable, some are not, and the console tells you which.
- **CloudWatch alarms on quota utilisation** are supported for many quotas. Alarm at 80 percent, not at failure.
- A **quota request template** applies requested increases automatically to **new accounts** in the organisation.
- Increases are processed by AWS on their timescale. Module 17 covered what that means during a disaster recovery test.

> ON THE EXAM
>
> Quotas fail silently until they fail completely. Alarming on utilisation turns a future outage into a support ticket you raise calmly.

Tagging as an operational control

- **Tag policies** in Organizations enforce allowed keys, allowed values and casing. Without them, CostCentre and costcenter coexist.
- **Cost allocation tags** must be **activated in the Billing console** before they appear in cost reports, and they are not retroactive.
- **Attribute-based access control** uses tags in IAM conditions, so one policy governs many resources without listing them.
- Tags drive **Backup selections, State Manager targets, Run Command targets and Config rules**. They are wiring, not labels.

> ON THE EXAM
>
> Almost every automation in this course targets by tag. A tag policy is what stops that automation quietly missing resources.

A Lambda function triggered asynchronously by S3 fails permanently for about 2 percent of objects because of a malformed file format. The team discovers this weeks later. There is no record of which objects failed, and the function has no destination or dead-letter queue configured.

The team needs failed events captured with enough context to reprocess them.

What should the team configure?

A. An on-failure destination on the function, targeting an SQS queue

B. Increase the function's timeout and retry attempts

C. Enable S3 versioning on the source bucket

D. Switch the trigger to synchronous invocation

◆ PAUSE AND DECIDE

What should the team configure?

A On-failure destination

Captures the event with request and response context after retries are exhausted, so failures are visible and reprocessable.

B More retries

The failures are permanent. Retrying a malformed file more times produces the same result more slowly.

C Bucket versioning

Protects the objects, which were never at risk. It says nothing about which invocations failed.

D Synchronous invocation

S3 notifications to Lambda are asynchronous by design. It would also mean no retries at all.

THE SIGNAL

"No record of which objects failed" is the clause. Asynchronous invocation with nothing configured discards the event silently, leaving only a metric.

> During a marketing campaign, a Lambda function behind API Gateway returns throttling errors for about ninety seconds. CloudWatch shows ConcurrentExecutions peaking at roughly 300 against an account limit of 1,000, and a matching spike in Throttles. The graph shows concurrency rising in distinct steps.
>
> No other function in the account was busy.

What is the cause?

A The account concurrency limit was reached

B The function hit Lambda's scaling rate, which limits how fast new execution environments are created

C Reserved concurrency was set too low on the function

D The function's timeout is too short

◆ PAUSE AND DECIDE

What is the cause?

A. Account limit reached

The scenario states 300 against a limit of 1,000, and no other function was busy. The pool was 70 percent free.

B. The scaling rate

Lambda creates new environments at a fixed rate. A sharp spike hits that ceiling regardless of how much pool capacity is unused, producing the stair-step pattern.

C. Reserved concurrency too low

Would cap the function at a fixed number, producing a flat ceiling rather than a rising staircase. Nothing suggests one was set.

D. Timeout too short

Produces timeout errors in the function's own logs, not throttling. Different failure entirely.

THE SIGNAL

Two numbers and one graph shape. 300 of 1,000 rules out the pool; the stair-step is the documented signature of the scaling rate.

> Three teams need to react to uploads in the same S3 bucket. The analytics team wants objects under data/, the compliance team wants everything, and the imaging team wants files ending in .jpg. All three need the ObjectCreated event.
>
> An engineer configures three S3 event notifications and the third is rejected.

What should the engineer do?

A Enable EventBridge notifications on the bucket and create three EventBridge rules

B Create three buckets, one per team

C Send one notification to an SNS topic and have each team subscribe

D Use S3 Object Lambda to route the events

◆ PAUSE AND DECIDE

What should the engineer do?

A Route to EventBridge

Removes the overlapping-filter limitation entirely, gives each team its own rule with richer filtering, and adds archive and replay.

B Three buckets

Solves a configuration limitation by duplicating storage and forcing uploaders to know which team cares. Unworkable at scale.

C One notification to SNS

The strongest alternative. It works, and every subscriber receives every event and must filter for itself, which is what EventBridge does natively.

D S3 Object Lambda

Transforms objects as they are retrieved. It has nothing to do with event routing.

THE SIGNAL

The rejection is the clue. S3 refuses overlapping prefix or suffix configurations for the same event type, and EventBridge has no such restriction.

> An SQS queue is processed by a Lambda function. During a backlog, the function scales to 900 concurrent executions. That exhausts the connection pool on the Aurora cluster behind it, causing errors across other applications sharing the database.
>
> The team must cap this function's database load without reducing capacity available to other Lambda functions in the account.

What should the team configure?

A MaximumConcurrency on the SQS event source mapping

B Reserved concurrency on the function

C Provisioned concurrency on the function

D Increase the account concurrency quota

◆ PAUSE AND DECIDE

What should the team configure?

A MaximumConcurrency on the event source mapping

Caps how many invocations the SQS poller drives, without reserving or removing anything from the shared account pool.

B Reserved concurrency

The near-miss. It does cap the function, and reserved capacity is subtracted from the shared pool, reducing what other functions can use.

C Provisioned concurrency

Pre-warms environments to remove cold starts. It does nothing to limit how many run, and would make matters worse.

D Raise the account quota

Allows even more concurrency, which is the opposite of what the database needs.

THE SIGNAL

The second clause is the discriminator. Both A and B cap the function; only A leaves the shared pool untouched.

> An operations team runs a nightly job that processes about 400,000 records from S3. It currently runs as one Lambda function that loops, and regularly hits the 15-minute timeout. Each record takes about two seconds and records are independent.
>
> The team needs error handling per record, and a report of which records failed.

What should the team use?

A. A Step Functions Distributed Map reading from S3, with Retry and Catch on the child workflow

B. Increase the Lambda timeout beyond 15 minutes

C. An SQS queue with one message per record and a Lambda consumer

D. An EC2 instance running the job as a script

◆ PAUSE AND DECIDE

What should the team use?

A Distributed Map with Retry and Catch

Reads the dataset from S3, fans out to thousands of parallel child executions, handles errors per record declaratively, and reports results.

B Longer timeout

Fifteen minutes is Lambda's hard maximum and cannot be raised. 400,000 records at two seconds each is over nine days serially anyway.

C SQS with one message per record

A reasonable design that does parallelise the work. Per-record error reporting means building it yourself from the DLQ.

D An EC2 instance

Runs serially unless you write the concurrency, and reintroduces a server to patch for a job that runs once a night.

THE SIGNAL

The arithmetic settles it before any service knowledge: 400,000 records at two seconds is over nine days of serial work.

> An order-processing Lambda function reads from a standard SQS queue. Occasionally a customer is charged twice. Investigation shows the function completed successfully but took slightly longer than the queue's visibility timeout, so the message was delivered again.
>
> The team cannot easily shorten the processing time.

Which two actions together address this? (Choose two.)

A. Increase the visibility timeout to comfortably exceed the processing time

B. Make the handler idempotent using an idempotency key recorded in DynamoDB

C. Switch the queue to a FIFO queue and accept the throughput reduction

D. Reduce maxReceiveCount to 1 so messages are never redelivered

◆ PAUSE AND DECIDE

Which two actions together address this?

A Raise the visibility timeout

Removes the immediate cause. A message must stay invisible for longer than the work takes, or it is redelivered while still in progress.

B Make the handler idempotent

Standard queues are at-least-once by design, so duplicates remain possible whatever the timeout. Idempotency makes them harmless.

C Switch to FIFO

Gives exactly-once processing and cuts throughput substantially, and it still requires the visibility timeout to exceed processing time.

D maxReceiveCount of 1

Sends a message to the DLQ after one failure. It does not prevent redelivery of an in-flight message, and it discards genuine retries.

THE SIGNAL

Fix the cause and survive the class of problem. A standard queue will deliver twice eventually, whatever you tune.

The five things to carry forward

- **Retry behaviour depends on the invocation model.** Asynchronous with no destination discards failures silently.
- **Reserved concurrency is a floor and a ceiling**, and it removes capacity from the shared pool.
- **The scaling rate is a second limit.** You can be throttled with most of the pool free.
- **S3 forbids overlapping notification filters.** Route the bucket to EventBridge instead.
- **Alarm on quota utilisation and on message age**, not only on failure and depth.

> ON THE EXAM
>
> That completes domain three. Next: IAM mechanics, and the start of domain four.